Unidad 2

Caso de estudio

Tecnología en Desarrollo de Software

Institución Universitaria Digital de Antioquia

Caso de estudio: procesamiento funcional de datos para un sistema de gestión de transporte urbano.

Contexto General

La Alcaldía de la ciudad de TecnoValle, una ciudad latinoamericana en pleno proceso de transformación digital, ha decidido modernizar su sistema de transporte urbano. Actualmente, la movilidad representa uno de los mayores desafíos, pues cada día se mueven más de 450.000 pasajeros a través de buses, rutas alimentadoras y estaciones principales.

Durante años, el sistema ha operado con una plataforma construida en un enfoque tradicional, imperativo y altamente acoplado. Esta plataforma presenta múltiples problemas:

- Retrasos en la visualización de información en tiempo real.
- Limitada capacidad para procesar grandes volúmenes de datos de pasajeros.
- Dificultad para detectar patrones y generar estadísticas diarias o semanales.
- Costo elevado de mantenimiento debido al código extenso, mutable y propenso a errores.

Con el crecimiento demográfico y la necesidad de decisiones basadas en datos, la alcaldía creó el proyecto TecnoMovil Data, cuyo objetivo es evolucionar el sistema hacia una arquitectura más modular, declarativa y escalable, apoyándose en principios de programación funcional.

1. Situación actual del sistema

Toda la información del transporte es recopilada desde:

- Validadores electrónicos de tarjetas.
- Sensores de entrada y salida por estación.
- Aplicaciones móviles de los usuarios.

- Cámaras y sensores IoT instalados en buses.

Cada día se generan entre 6 y 12 millones de registros, representando:

- Número de pasajeros por ruta.
- Horas pico y horas valle.
- Sobreocupación o subutilización de buses.
- Patrones de viaje por usuario.
- Kilometraje recorrido y tiempos de viaje.

El sistema actual, basado en programación imperativa, procesa los datos de forma secuencial, lo que provoca:

- Procesamiento lento (hasta 2 horas por día).
- Accesos mutables a estructuras compartidas.
- Error en los conteos por condiciones de carrera.
- Código extenso y difícil de mantener.
-

Los ingenieros del proyecto han concluido que el módulo más crítico para rediseñar es el encargado del procesamiento y agregación de datos diarios.

2. Desafío para los estudiantes

Diseñar un módulo funcional en Java usando Streams, Lambdas, funciones puras y colecciones inmutables, capaz de procesar un conjunto grande de datos simulados.

Este módulo se integrará al nuevo sistema TecnoMovil Data.

3. Requerimientos funcionales del caso de estudio

Desarrollar, en Java, un conjunto de operaciones funcionales que respondan a las siguientes necesidades:

3.1 Estructura de datos a Procesar

Cada registro está compuesto por:

- idUsuario
- ruta
- estación
- accion // "entrada" o "salida"
- timestamp

Ejemplo de acciones:

- "entrada" cuando el usuario ingresa a la estación.
- "salida" cuando valida para salir o cambia de ruta.

3.2 Tareas a implementar (solo se describen, NO se resuelven)

Los estudiantes deben implementar mediante programación funcional:

a) Cálculo de afluencia por estación

   Contar cuántos usuarios ingresan a cada estación durante el día.

b) Identificación de horas pico

   Agrupar los registros por hora del día y calcular momentos de mayor flujo.

c) Rutas más utilizadas

Determinar cuáles rutas tienen mayor volumen de uso.

d) Patrones de viaje por usuario

Generar, para cada usuario, un listado de estaciones visitadas en el orden en que aparecen en los registros.

e) Cálculo de tiempo promedio entre estaciones

Usar operaciones funcionales para estimar el tiempo que toman los usuarios en moverse entre estaciones consecutivas.

f) Detección de sobrecarga en rutas

Si una ruta supera un umbral de ocupación (simulado), marcarla como "crítica".

Consideraciones técnicas:

- Se deben usar funciones puras para el procesamiento de registros.
- No deben modificarse listas originales (inmutabilidad).
- Deben emplearse lambdas y pipelines con Streams.
- Las colecciones deben tratarse desde un enfoque declarativo.
- El análisis debe permitir paralelización si se decide usar parallelStream().
- La construcción de las operaciones debe evitar totalmente efectos secundarios.



Institución Universitaria Digital de Antioquia



Esta licencia permite a otros distribuir, remezclar, retocar, y crear a partir de esta obra de manera no comercial y, a pesar que sus nuevas obras deben siempre mencionar a la IU Digital y mantenerse sin fines comerciales, no están obligadaos a licenciar obras derivadas bajo las mismas condiciones.

www.iudigital.edu.co